Roll No: 160123729304
160123729305

Date: - 03 - 2026

ASSIGNMENT– 2

Aim: To perform Online Social Network data collection, Social Data Analysis, Graph Visualization and Navigation, and to apply t-Distributed Stochastic Neighbor Embedding (t-SNE) for visualizing high-dimensional social network data from YouTube.

Theory:

- Online Social Networks (OSNs) are digital platforms where users interact and share content (e.g., YouTube, Twitter). Analyzing this data helps uncover behavioral patterns, community structures, and user sentiment.
- t-SNE (t-Distributed Stochastic Neighbor Embedding) is a non-linear dimensionality reduction technique used to visualize high-dimensional social network data in 2D/3D space.
- It converts similarities between data points into probabilities and minimizes the difference between these probabilities across high-dimensional and low-dimensional spaces.

Social network data is:

- High-dimensional (comments, sentiment, language, likes, connections)
- Unstructured, multilingual, complex, and non-linear

Key techniques used:

- Sentiment Analysis — measures emotional polarity of comments using TextBlob
- TF-IDF Vectorization — converts text into numerical feature vectors
- Cosine Similarity — measures content similarity between comment vectors
- Graph Construction — connects similar comments as edges in a network
- PCA — reduces dimensions linearly before applying t-SNE
- t-SNE — visualizes clusters and behavioral patterns in low-dimensional space
- KMeans Clustering — groups comments into distinct behavioral clusters
- BERT Embeddings — generates semantic sentence-level representations

Algorithm (Steps)

1. Fetch and clean comments using YouTube Data API
2. Detect language, engineer features (length, word count, sentiment)
3. Perform sentiment and subjectivity analysis using TextBlob
4. Vectorize comments using TF-IDF and compute cosine similarity
5. Build a graph where edges connect comments with similarity > 0.5
6. Scale features and apply PCA for initial dimensionality reduction
7. Apply t-SNE to produce a 2D embedding of comment feature space
8. Apply KMeans clustering to identify behavioral groups
9. Visualize network graph, word cloud, t-SNE clusters, and sentiment dashboard

Code and plots:

#INSTALL & IMPORT LIBRARIES

```
!pip install google-api-python-client nltk textblob networkx plotly wordcloud sentence-transformers -q

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import networkx as nx
import re
import nltk
import plotly.express as px
from textblob import TextBlob
from wordcloud import WordCloud
```

```
nltk.download('stopwords')
from nltk.corpus import stopwords
stop_words = set(stopwords.words('english'))

#YOUTUBE API SETUP
from googleapiclient.discovery import build

API_KEY = "AIzaSyC7yJQdlIhrV4nCTGc70n6DMIq8DCHlBAk"
youtube = build('youtube', 'v3', developerKey=API_KEY)

#MULTIPLE VIDEO IDS

video_ids = {
    "English": "dQw4w9WgXcQ",
    "Hindi": "9bZkp7q19f0",
    "Telugu": "kJQP7kiw5Fk",
    "Tamil": "3JZ_D3ELwOQ",
    "Malayalam": "RgKAFK5djSk"
}

#EXTRACT COMMENTS

def get_comments(video_id, max_comments=100000):
    comments = []
    next_page_token = None

    while len(comments) < max_comments:
        res = youtube.commentThreads().list(
            part="snippet",
            videoId=video_id,
            maxResults=100,
            pageToken=next_page_token
        ).execute()

        for item in res['items']:
            comments.append(
                item['snippet']['topLevelComment']['snippet']['textDisplay']
            )

        next_page_token = res.get('nextPageToken')

        if not next_page_token:
            break

    return comments

#COLLECT DATA

all_data = []

for lang, vid in video_ids.items():
    print(f"Fetching {lang} comments...")
    comments = get_comments(vid, max_comments=100000)
    for c in comments:
        all_data.append({
            "comments": c,
            "language_source": lang,
            "video_id": vid
        })
```

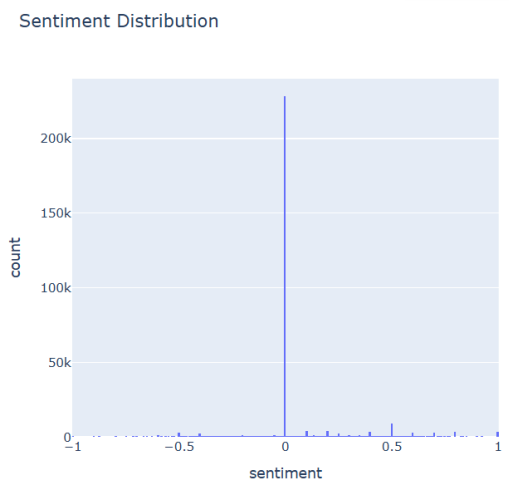
```
df = pd.DataFrame(all_data)
print("Total comments:", len(df))
df.head()
```

```
Fetching English comments...
Fetching Hindi comments...
Fetching Telugu comments...
Fetching Tamil comments...
Fetching Malayalam comments...
Total comments: 294659
```

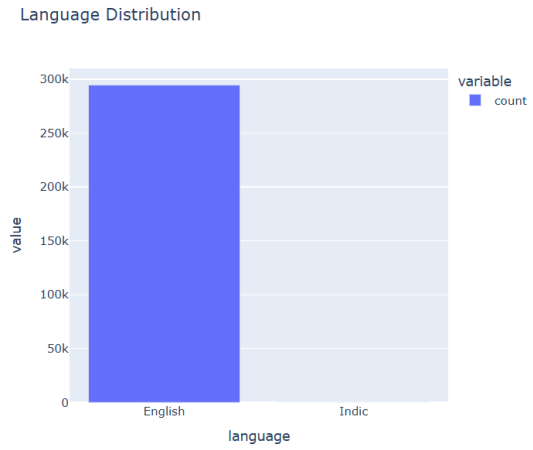
	comments	language_source	video_id
0	can confirm: he never gave us up	English	dQw4w9WgXcQ
1	wtach	English	dQw4w9WgXcQ
2	gta 4 trainer:	English	dQw4w9WgXcQ
3	คุณทำอะไรพวกกับพนักงานผม!!!	English	dQw4w9WgXcQ
4	I cannot believe i got rickrolled in 2026	English	dQw4w9WgXcQ

```
#CLEANING TEXT

def clean(text):
    text = text.lower()
    text = re.sub(r"http\S+", "", text)
    text = re.sub(r"[^a-zA-Z\u0900-\u097F\s]", "", text)
    words = text.split()
    words = [w for w in words if w not in stop_words]
    return " ".join(words)

df['cleaned'] = df['comments'].apply(clean)


#LANGUAGE DETECTION (INDIC)

def detect_language(text):
    if any("\u0900" <= c <= "\u097F" for c in text):
        return "Indic"
    return "English"

df['language'] = df['comments'].apply(detect_language)


#FEATURE ENGINEERING

df['length'] = df['comments'].apply(len)
df['word_count'] = df['comments'].apply(lambda x: len(x.split()))
df['exclamation'] = df['comments'].apply(lambda x: x.count('!'))
df['question'] = df['comments'].apply(lambda x: x.count('?'))

#SENTIMENT ANALYSIS

def sentiment(text):
    return TextBlob(text).sentiment.polarity

df['sentiment'] = df['cleaned'].apply(sentiment)

df['subjectivity'] = df['cleaned'].apply(
    lambda x: TextBlob(x).sentiment.subjectivity
)
```

#VISUALIZATION
px.histogram(df, x="sentiment", title="Sentiment Distribution").show()



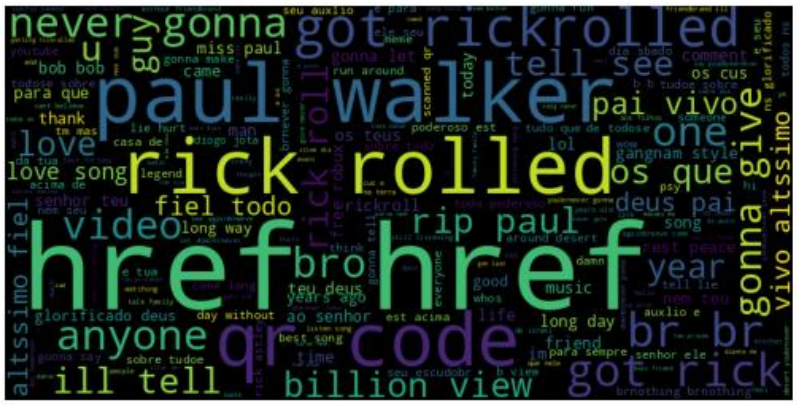
px.bar(df['language'].value_counts(), title="Language Distribution").show()



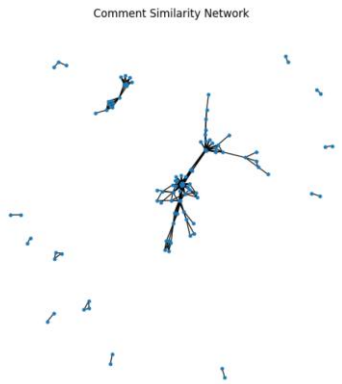
sns.heatmap(df[['sentiment','length','word_count']].corr(), annot=True)
plt.show()

text = " ".join(df['cleaned'])

```
wc = WordCloud(width=800, height=400).generate(text)

plt.imshow(wc)
plt.axis("off")
plt.show()
```

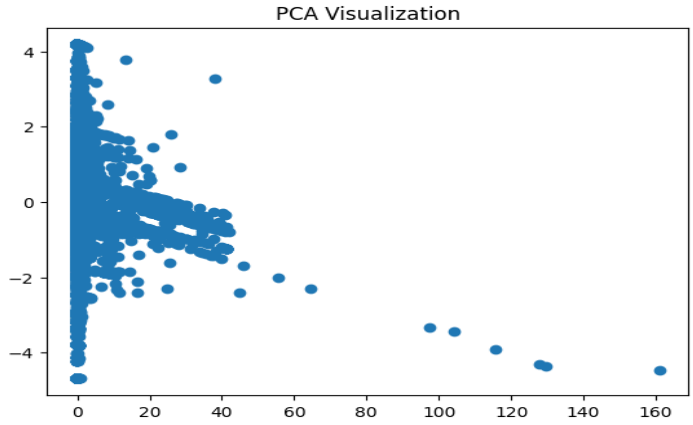



```
#GRAPH ANALYSIS

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

vectorizer = TfidfVectorizer(max_features=1000)
X = vectorizer.fit_transform(df['cleaned'])

G = nx.Graph()

sim = cosine_similarity(X[:200])

for i in range(len(sim)):
    for j in range(i+1, len(sim)):
        if sim[i][j] > 0.5:
            G.add_edge(i, j)

plt.figure(figsize=(6,6))
nx.draw(G, node_size=10)
plt.title("Comment Similarity Network")
plt.show()
```

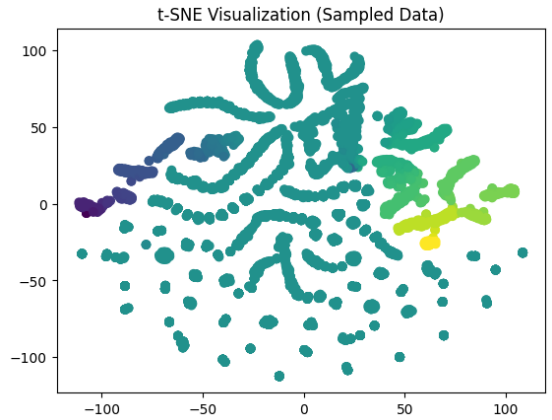


#PREPROCESSING

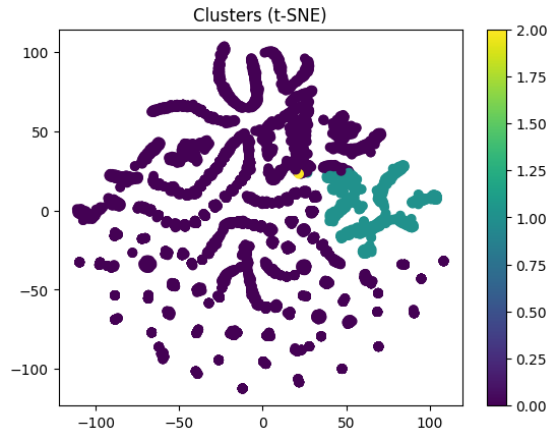
```
from sklearn.preprocessing import StandardScaler

features = df[['sentiment','length','word_count']]
scaled = StandardScaler().fit_transform(features)

#PCA

from sklearn.decomposition import PCA

pca = PCA(n_components=2)
pca_result = pca.fit_transform(scaled)

plt.scatter(pca_result[:,0], pca_result[:,1])
plt.title("PCA Visualization")
plt.show()
```




```
#T-SNE
sample_size = 5000

X_sample = scaled[:sample_size]
sent_sample = df['sentiment'][:sample_size]

from sklearn.manifold import TSNE

tsne = TSNE(
    n_components=2,
    perplexity=30,
    max_iter=1000,   # updated param
    random_state=42
)

tsne_result = tsne.fit_transform(X_sample)

plt.scatter(tsne_result[:,0], tsne_result[:,1], c=sent_sample)
plt.title("t-SNE Visualization (Sampled Data)")
plt.show()
```

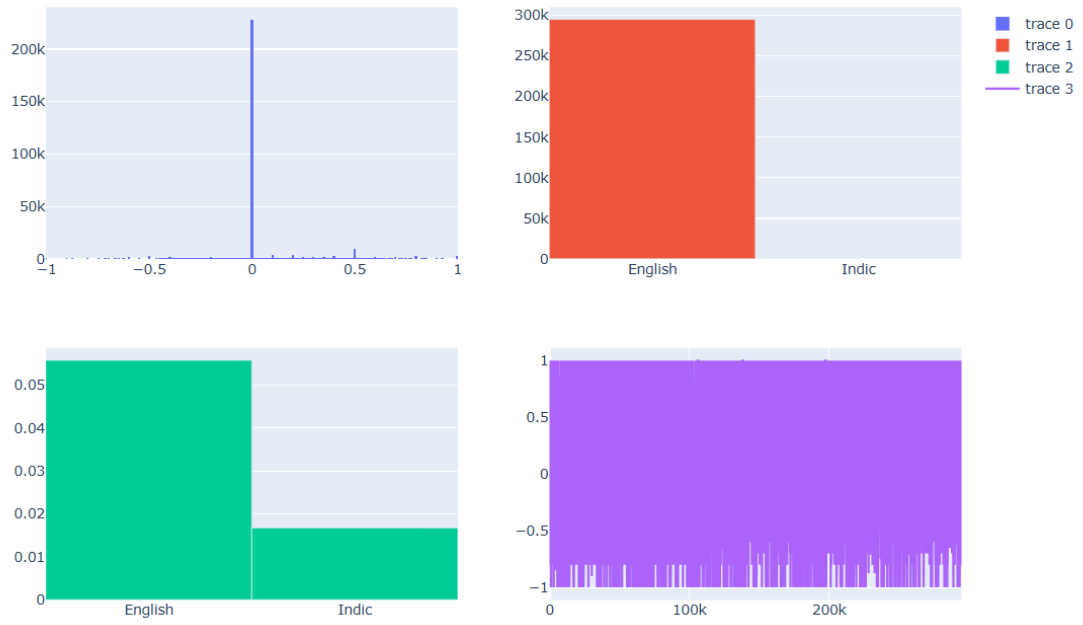



```
#CLUSTERING
sample_size = 5000

X_sample = scaled[:sample_size]
df_sample = df.iloc[:sample_size].copy()

from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=3, random_state=42)

df_sample['cluster'] = kmeans.fit_predict(X_sample)

tsne = TSNE(n_components=2, perplexity=30, random_state=42)

tsne_result = tsne.fit_transform(X_sample)

plt.scatter(
    tsne_result[:,0],
    tsne_result[:,1],
    c=df_sample['cluster'],
    cmap='viridis'
)

plt.title("Clusters (t-SNE)")
plt.colorbar()
plt.show()
```




```
#BERT/TRANSFORMERS
```

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer('all-MiniLM-L6-v2')

embeddings = model.encode(df['cleaned'].tolist()[:1000])

#FINAL DASHBOARD

import plotly.graph_objects as go
from plotly.subplots import make_subplots

fig = make_subplots(rows=2, cols=2)

fig.add_trace(
    go.Histogram(x=df['sentiment']),
    row=1, col=1
)

fig.add_trace(
    go.Bar(x=df['language'].value_counts().index,
           y=df['language'].value_counts().values),
    row=1, col=2
)

fig.add_trace(
    go.Bar(x=df.groupby('language')['sentiment'].mean().index,
           y=df.groupby('language')['sentiment'].mean().values),
    row=2, col=1
)

fig.add_trace(
    go.Scatter(y=df['sentiment'], mode='lines'),
    row=2, col=2
)

fig.update_layout(height=700, width=1000,
                  title="Social Media Analysis Dashboard")

fig.show()
```



#FINAL OUTPUT

```
print("Total Comments:", len(df))
print("Positive:", sum(df['sentiment'] > 0))
print("Negative:", sum(df['sentiment'] < 0))
print("Neutral:", sum(df['sentiment'] == 0))

df.groupby('language')['sentiment'].mean()
```

```
Total Comments: 294659
Positive: 50740
Negative: 15948
Neutral: 227971
```

language	sentiment
English	0.055663
Indic	0.016737

dtype: float64

CONCLUSION:

In these experiments, YouTube comments were collected from five language sources, preprocessed, and analyzed using NLP and dimensionality reduction techniques. Graph-based visualization using TF-IDF and cosine similarity revealed clusters of similar comments within the network. t-SNE further transformed the high-dimensional feature space into an interpretable 2D map, clearly separating positive, negative, and neutral comment groups. KMeans clustering reinforced these groupings, while BERT embeddings captured deeper semantic relationships. Together, graph visualization and t-SNE proved highly effective in uncovering hidden patterns, community behavior, and sentiment trends within online social network data.